

Compiler Construction: Practical Introduction

(Almost) no theory
but a lot of code

A lot of information
is outside of the course

Lecture 1 Introduction to the field

Eugene Zouev
Fall Semester 2025
Innopolis University

Who Is This Guy? 😊

- Eugene Zouev / Евгений Зуев
- Have been working at Moscow Univ., Swiss Fed Inst of Technology (ETH Zürich), EPFL (Lausanne); PhD (1999, Moscow Univ).
- Prof. interests:
compiler construction, language design, PL semantics.
- The author of the 1st Russian C++ front-end compiler
- Interstron Ltd., Moscow, 1999-2000.
- Zonnon language implementation for .NET & Visual Studio
- ETH Zürich, 2005.
- Swift prototype compiler for Tizen & Android
- Samsung Research, 2015
- Six (or seven? 😊) books; the latest are
 - «Редкая профессия», ДМК Пресс, Москва 2014.
 - Software Design for Resilient Computer Systems, Springer, 2019



Before we start...

A remark about language
syntax & semantics

“Usual” view at a language:

Syntax:

A set of rules that regulate
the structure of programs
and their parts (constructs)

Semantics:

The meaning of the constructs

Static semantics:

- How programs get compiled

Dynamic semantics:

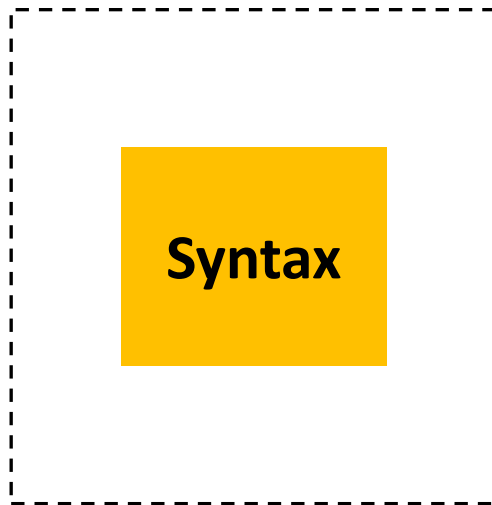
- How programs get executed.



Before we start...

A remark about language
syntax & semantics

Reality



A diagram illustrating the relationship between Syntax and Semantics. It consists of a large yellow square labeled 'Semantics' with a smaller dashed black square centered inside it.

Semantics

Conclusions for compiler developers:

- Language semantics is much more important and hard to implement.
- Assign much more time in your work on a compiler implementation to semantic analysis.

Why Compiler Course?

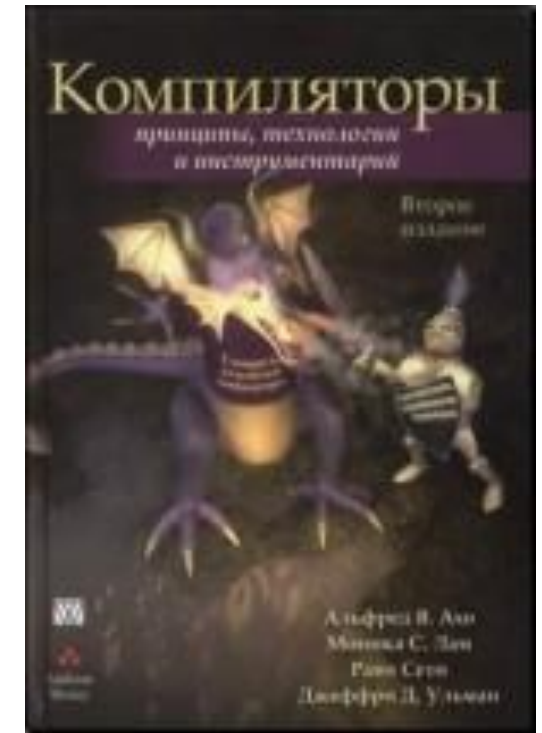
- The area of compiler construction is one of **three cornerstones** of Computer Science (two others are databases & operating systems).
- The area includes many **fundamental results**: formal grammar theory, type theory, graph theory, algorithm theory and many others.
- The similar courses are given in universities **all over the world**.
- It's the extremely interesting area 😊.

References (1)

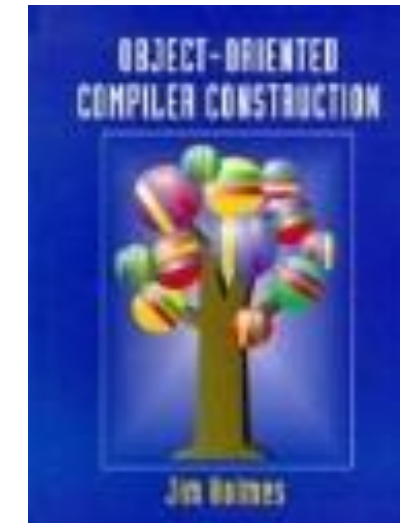
- Alfred V.Aho, Monica S.Lam, Ravi Sethi, Jeffrey D. Ullman, **Compilers. Principles, Techniques, & Tools**, Second Edition, Addison-Wesley, 2007, ISBN 0-321-48681-1. ("**Dragon book**")

Russian translation:

Ахо, Альфред В., Лам, Моника С., Сети, Рави, Ульман, Джеффри Д. Компиляторы: принципы, технологии и инструментарий, 2-е изд.: Пер. с англ.- М.: ООО «И.Д.Вильямс», 2008.- 1184 с.: илл. ISBN 978-5-8459-1350-4.



- Jim Holmes
Object-Oriented Compiler Construction, 1994



References (2)

- N.Wirth, **Compiler Construction**, Addison-Wesley, 1996, ISBN 0-201-40353-6;

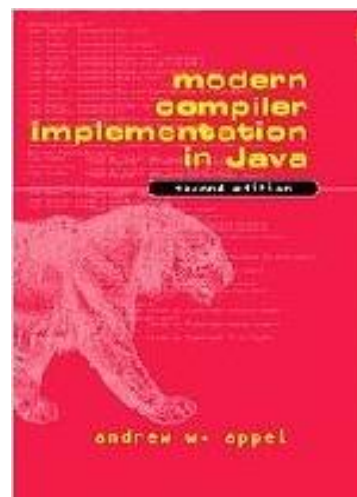
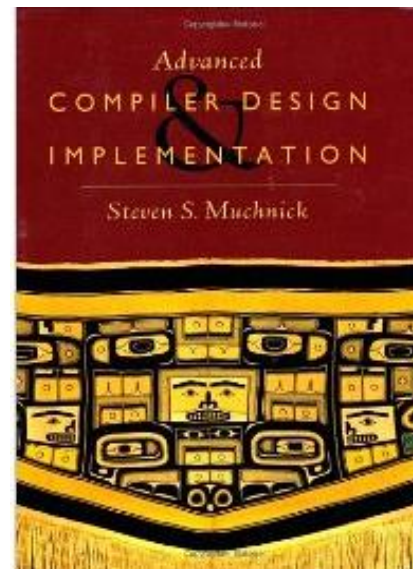
<http://www.ethoberon.ethz.ch/WirthPubl/CBEAll.pdf>, 2005.

Russian translation:

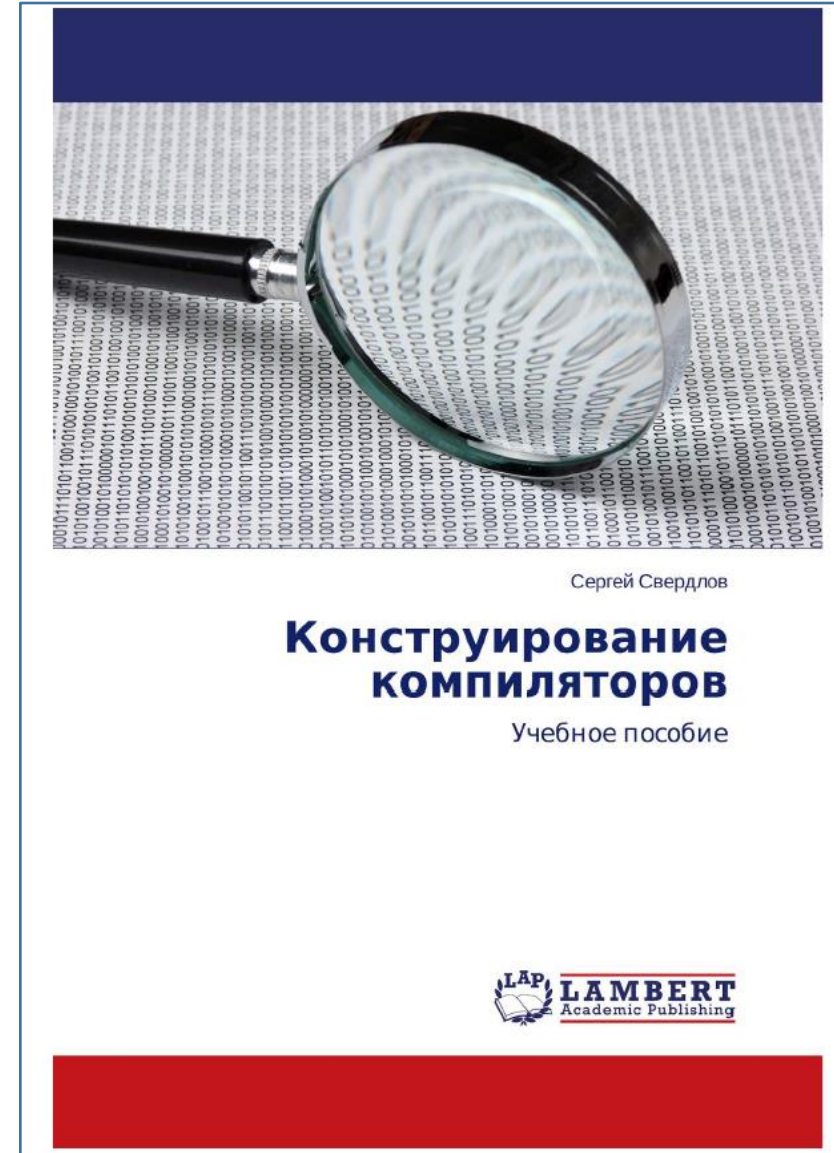
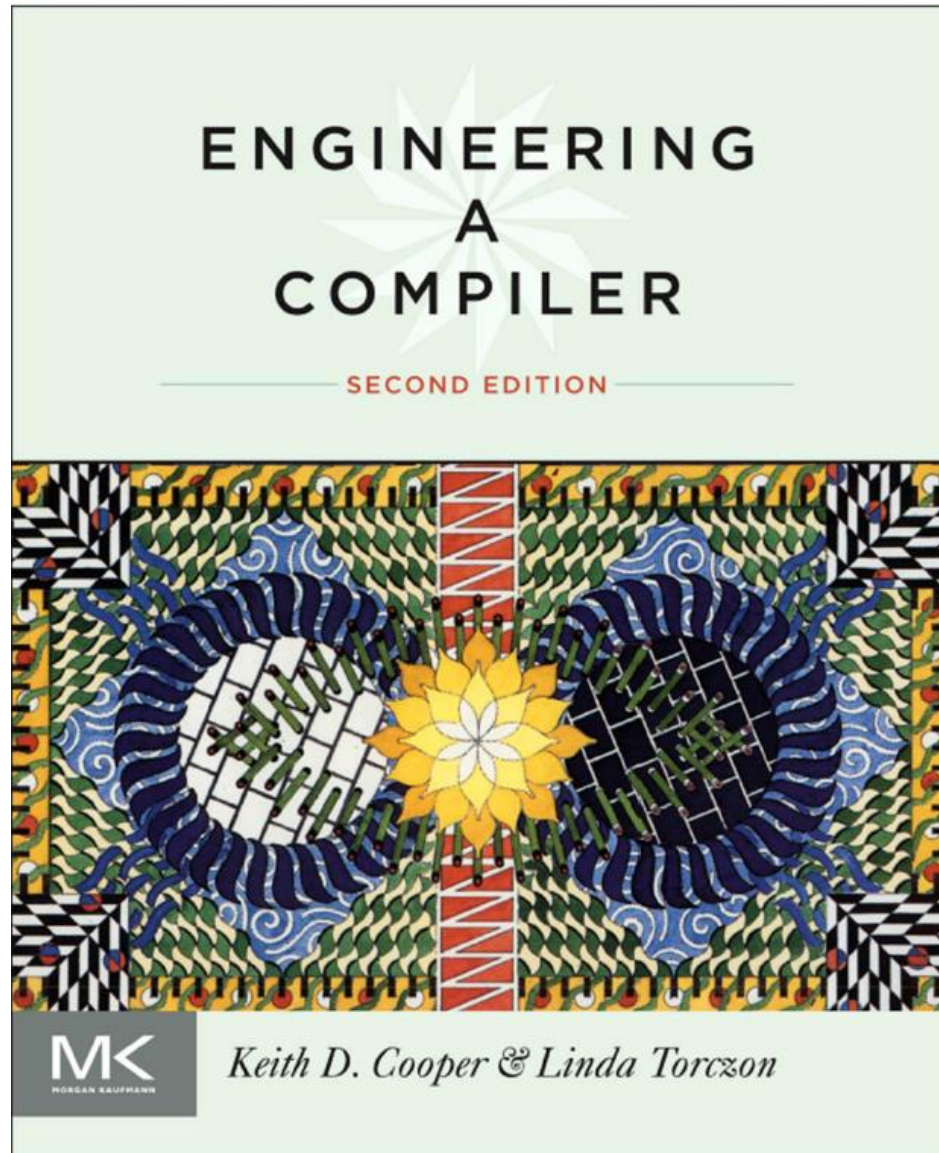
Никлаус Вирт, **Построение компиляторов**, ДМК-Пресс, 2010, ISBN 978-5-94074-585-3.

- S.Muchnik.
Advanced Compiler Design and Implementation 1997, ISBN 1-55860-320-4.

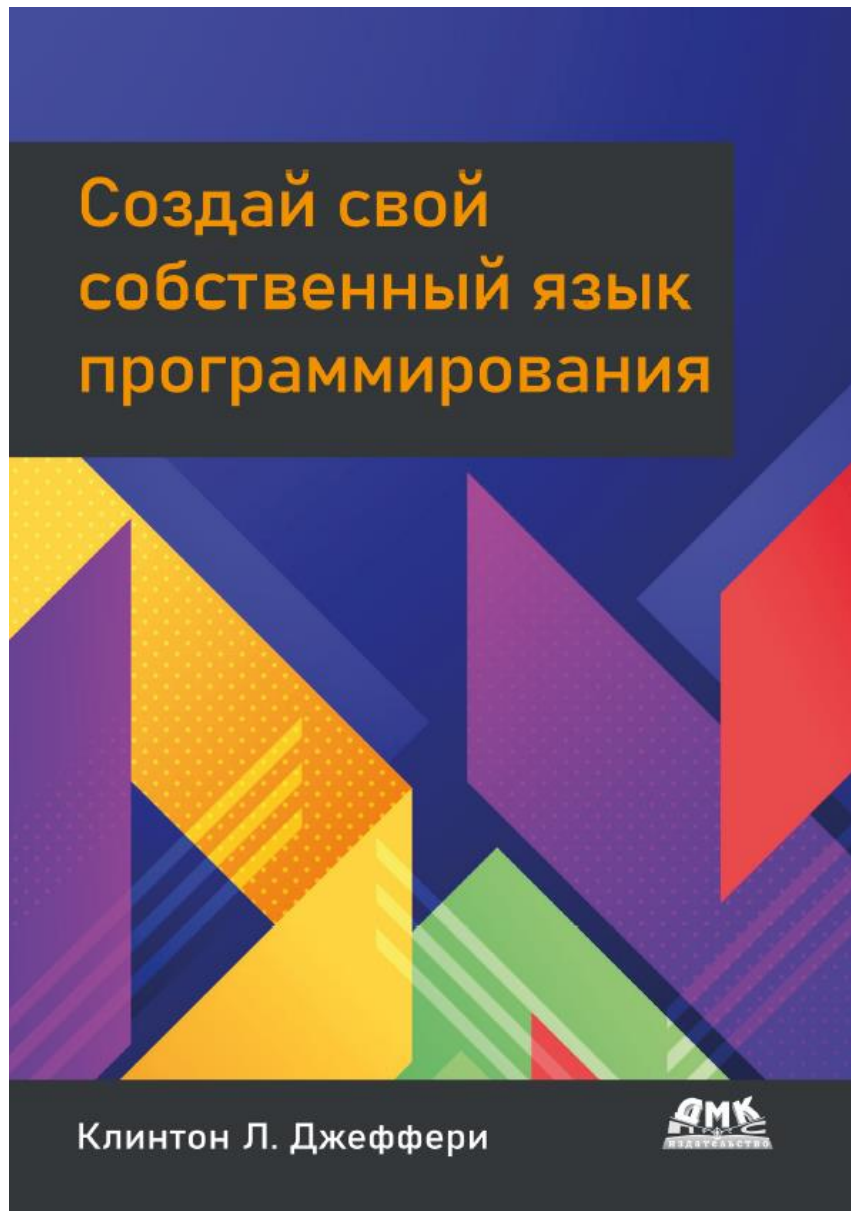
- Andrew Appel
Modern Compiler Implementation in {Java, C, ML} 1998



References (3)



References (4)



Many more books; some of them are available electronically

Список литературы для начинающих компиляторщиков
<https://github.com/true-grue/Compiler-Development/blob/master/docs/beginners.md>

Textbooks: Problems (1)

- A lot of theory but a few actual examples (even if there are many “exercises”).
- Many books in compilers (except some 😊) are not *practically-targeted*.
- The books consider **artificial**, “model” languages: small, simple, regular, “academic”. Real languages are much more complicated and typically have (very) bad design – hard to implement.

Typically, books do not discuss how **real language features** get implemented in compilers.

Textbooks: Problems (2)

- Language semantics is the most important part of any language (syntax is just “decoration”), and the major part of any compiler **deals with semantics, not with syntax**. However, textbooks consider syntactic aspects of languages and compilers very carefully, in contrast with semantic implementation issues...
- Some aspects are not presented in those books **at all**: e.g., development tools, error recovering, garbage collection, language-specific optimizations etc.

Extra Sources & Prerequisites

- Some knowledge of one or two programming languages is highly recommended (the more the better 😊)
 - **C**, **C++**, **C#**, **Java**, Pascal, Python, ...
- The preference is Object-Oriented paradigm.
- Some experience in working with modern IDEs is also needed.
 - Visual Studio, Eclipse, IDEA, NetBeans, CLion...

Extra information sources

- Magazine papers, PhD theses, conference presentations.
- Source codes of real compilers.
- **International standards for PLs.**
- Some extra books/papers.

The Structure of the Course

See Moodle
for all related info

- Fourteen lectures + the final event.
- Each Tuesday 12:40-14:10.
- Fourteen Labs/Tutorials (after each lecture)

Basic information

Project discussions, reports,
case studies, master solutions

The grading formula:

Lexical analyzer report	10%
Syntax analyzer report	10%
Sem. analyzer report	10%
CodeGen/Interp report	10%
Final project presentation	10%
Project results (quality, readiness etc.)	50%

- $\geq 90\%$ 5
- 80-89% 4
- 60-79% 3
- $\leq 59\%$ Fail

- No assignments
- No final exam
- **Projects instead!**

Projects are to be
presented on the first lab

The Structure of the Course

Week	Date	Class	Topics
1	Sept 1	Lecture	Introduction to the topic; Regulations. Compilation stages
		Tutorial 1	Project languages' presentations
2	Sept 8	Lecture 2	Lexical analysis
		Lab 2	Task description. Project architecture. Test suite.
3	Sept 15	Lecture 3	Syntax analysis
		Lab 3	First team reports: The lexical analyzer
4	Sept 22	Lecture 4	Compilation Structures
		Tutorial 4	Case study: Lexical analysis: master solution
5	Sept 29	Lecture 5	Parser generators: Yacc/Bison
		Tutorial 5	Case Study: The complete Bison-based Java parser
6	Oct 6	Lecture 6	Semantic Analysis
		Tutorial 6	Bootstrapping technology
7	Oct 7	Lecture 7	Source-level optimizations
		Lab 7	Second team reports: The syntax analyzer
8	Oct 13	Lecture 8	Compilation for Virtual Machines & .NET Architecture
		Tutorial 8	Case study: Syntax analysis: master solution
9	Oct 20	Lecture 9	<i>TBD</i>
		Lab 9	<i>TBD</i>
10	Oct 27	Lecture 10	Bytecode interpretation techniques
		Lab 10	Third team reports: Semantic analysis
11	Nov 3	Lecture 11	Python: Compiler & Virtual Machine
		Tutorial 11	Case study: Semantic analysis: master solution
12	Nov 10	Lecture 12	The LLVM infrastructure for Compiler Construction
		Tutorial 12	Case study: Code generation: master solution
13	Nov 17	Lecture 13	Introduction to the language runtime support
		Lab 13	Fourth team reports: Code generation OR interpretation
14	Nov 24	Lecture 14	The Evolution of the Compiler Architecture
		Tutorial 14	Case study: Virtual machine: master solution
15	Dec		Final project presentations

Compilation: The Goal & The Task

- **The Goal:**

To overcome the «semantic gap» between human's way of thinking and computer solving task defined by a human.

- **The Task:**

To transform a human-written and human-friendly ☺ program to a form which can be executed by a computer.

Semantic Gaps

Application domain

Train traffic management system:

Trains, velocity, distance, railways, switches, train stations, conflict resolution, time schedule etc.

Simulation of the Universe evolution:

Planet movement rules, planet systems, stars classification, star activity etc.

...

Computer program/ programming language

- Variable, array, ...
- Function, operator, procedure
- Control structures
- Data types
- Module, class, interface
- ...

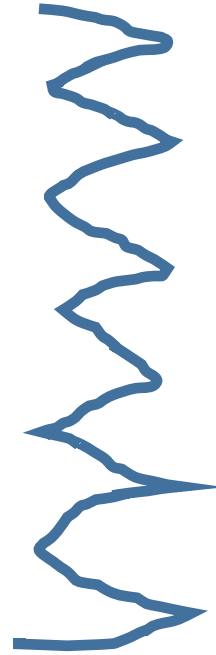
Computer hardware

- Memory cell
- Memory address
- Register
- Instruction, instruction set
- ...

Semantic Gap

Computer program/ programming language

- Variable, array, ...
- Function, operator, procedure
- Control structures
- Data types
- Module, class, interface
- ...



Computer hardware

- Memory cell
- Memory address
- Register
- Instruction, instruction set
- ...

So, the main compiler task is:

- To **map** human-written & human-friendly program to computer hardware - to solve a problem

OR:

- To overcome the semantic gap.

Why Compilers Matter?

Program source text

```
int main()
{
    Stack<double> stack1;
    Stack<int> stack2(5);
    int y = 1;
    double x = 1.1;
    int i, j;
    cout << "\n pushed values into stack1: ";
    for ( i=1; i<=11; i++)
    {
        if (stack1.push(i*x))
            cout << endl << i*x;
        else
            cout << "\n stack1 is full";
    }
    cout << "\n\n popd values from stack1:\n";
    for (i=1; i<=6; i++)
        cout << stack1.pop() << endl;
    ...
}
```

COMPILER

Machine code

```
0x006 77 22378EE
0x007 00 0000001
0x008 33 1017700
0x009 7B 00178AB
0x00A 7B 00178AB
0x00B 72 037CEFF
0x00C 3D AFFFFED
0x00E 72 037CEFF
0x00F 3D AFFFFED
0x00D 7B 00178AB
0x00E 3D CAFEBEB
0x00F 3D 00011FF
...
```

Execution

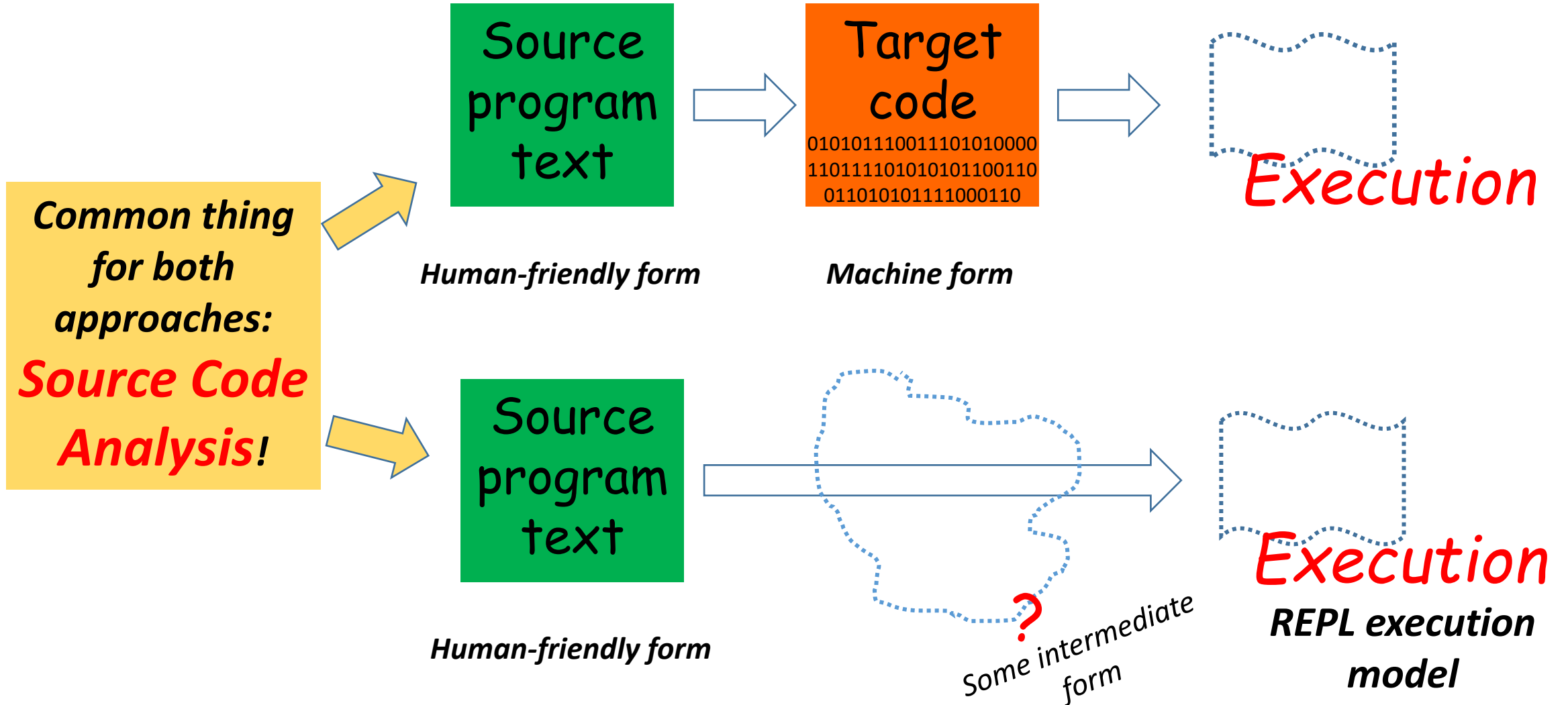
Is this a program? ☺

- **No**: this is just a text

This is a program

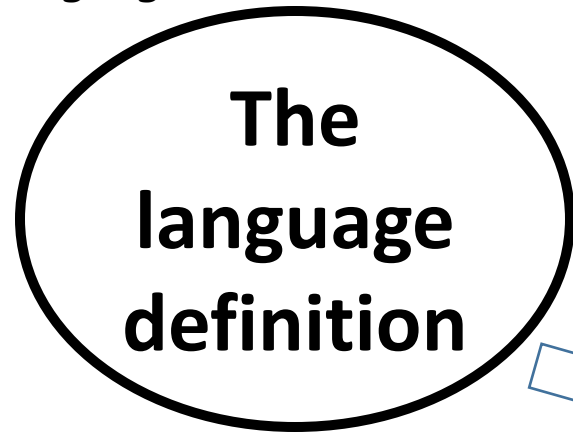
What's Inside???

Compilation vs Interpretation

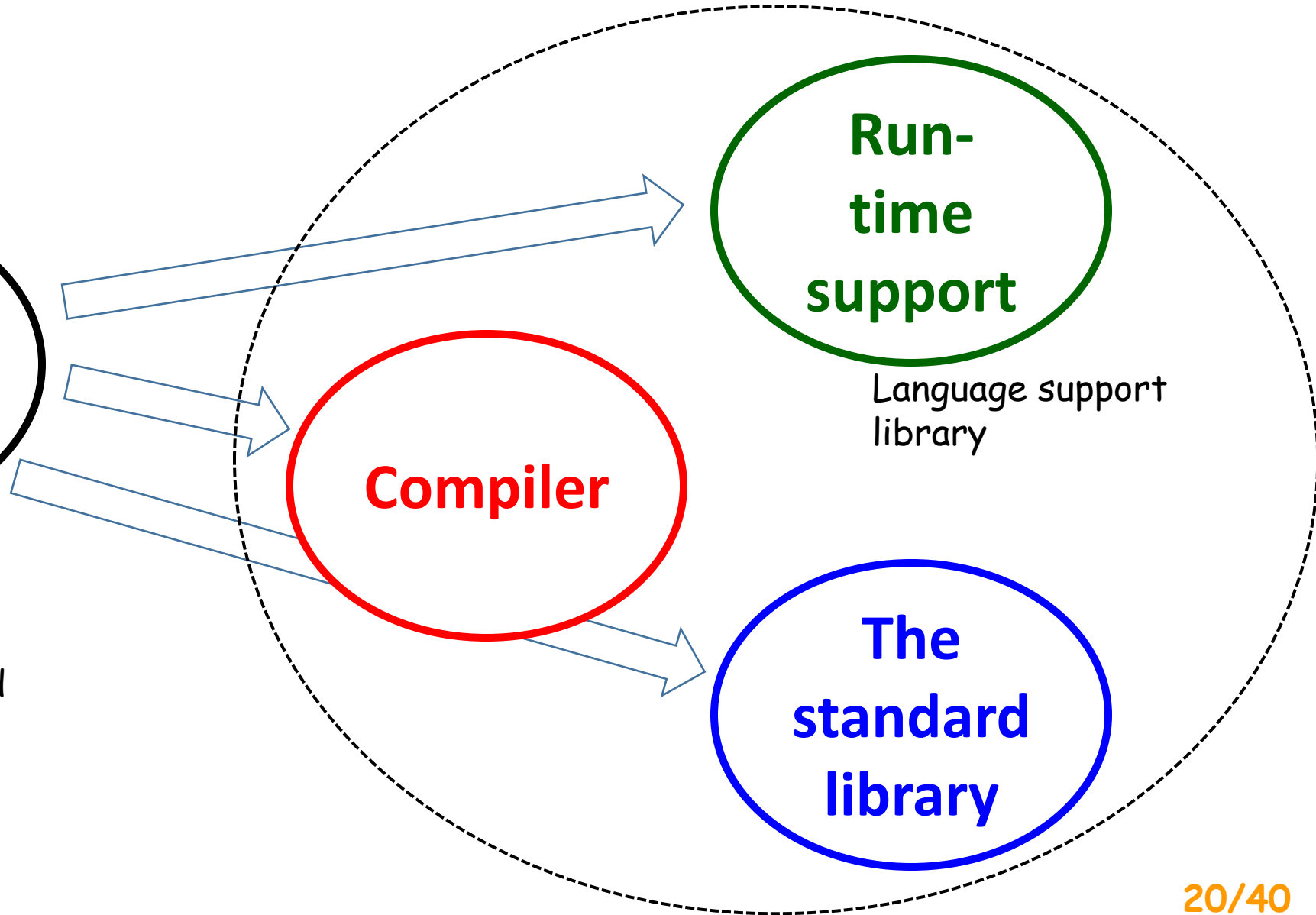


Compilation Infrastructure

The primary source of information about the language

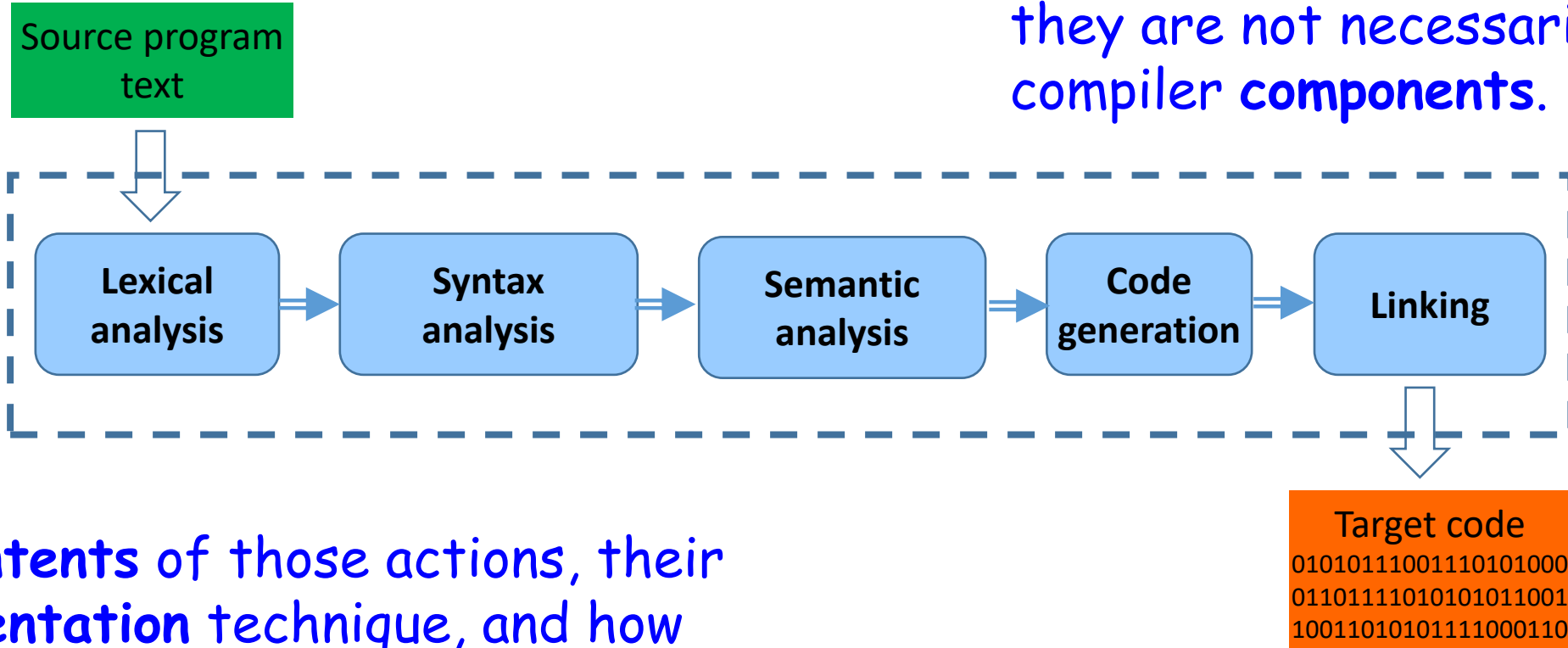


- Reference manual
- "Revised report"
- International standard
- Informal description
- etc.



Compilation: An Ideal Picture

*A program written by a human
(or by another program)*



Blue squares just denote some **actions** typical to any compiler; they are not necessarily actual compiler **components**.

The **contents** of those actions, their **implementation** technique, and how they **interact** with other actions - is just the subject of the course.

*A program binary image
suitable for immediate
execution by a machine*

Compilation stages: C# compiler

Reality

1. Primary lexical analysis (the result is a stream of tokens).
 2. High-level syntax analysis (method bodies are not processed but get stored).
 3. Declaration processing.
 4. Two passes checking cyclic type & generic dependencies.
 5. Two passes for some semantic checks and constant fields processing.
 6. Lexical analysis (!) of method bodies, constructors, properties and indexers.
 7. Type analysis in method bodies.
- Subsequent stages are actually AST tree "walkers" modifying the AST.
- Loop conversion: replacing loops for goto's & labels.
 - **Eight** passes looking for various types of errors.
 - **Fourteen** (!) passes for optimization and simplifying AST.
 - **Four** passes generating MSIL code.

Compilation Phases: C++

Reality

ISO/IEC 14882:2011(E) Programming Languages -- C++,
Third Edition (Excerpt from the section 2.2).

Initial source text analysis

1. Physical source file characters are mapped, in an implementation-defined manner, to the basic source character set...
2. Each instance of a backslash character (\) immediately followed by a new-line character is deleted, splicing physical source lines to form logical source lines...
3. The source file is decomposed into **preprocessing tokens**...

Preprocessing

4. **Preprocessing directives are executed**, macro invocations **are expanded**, and `_Pragma` unary operator expressions are executed...

Literal processing

5. Each source character set member in a character literal or a string literal, as well as each escape sequence and universal-character-name in a character literal or a non-raw string literal, **is converted** to the corresponding member of the execution character set...
6. Adjacent string literal tokens **are concatenated**.

Translation

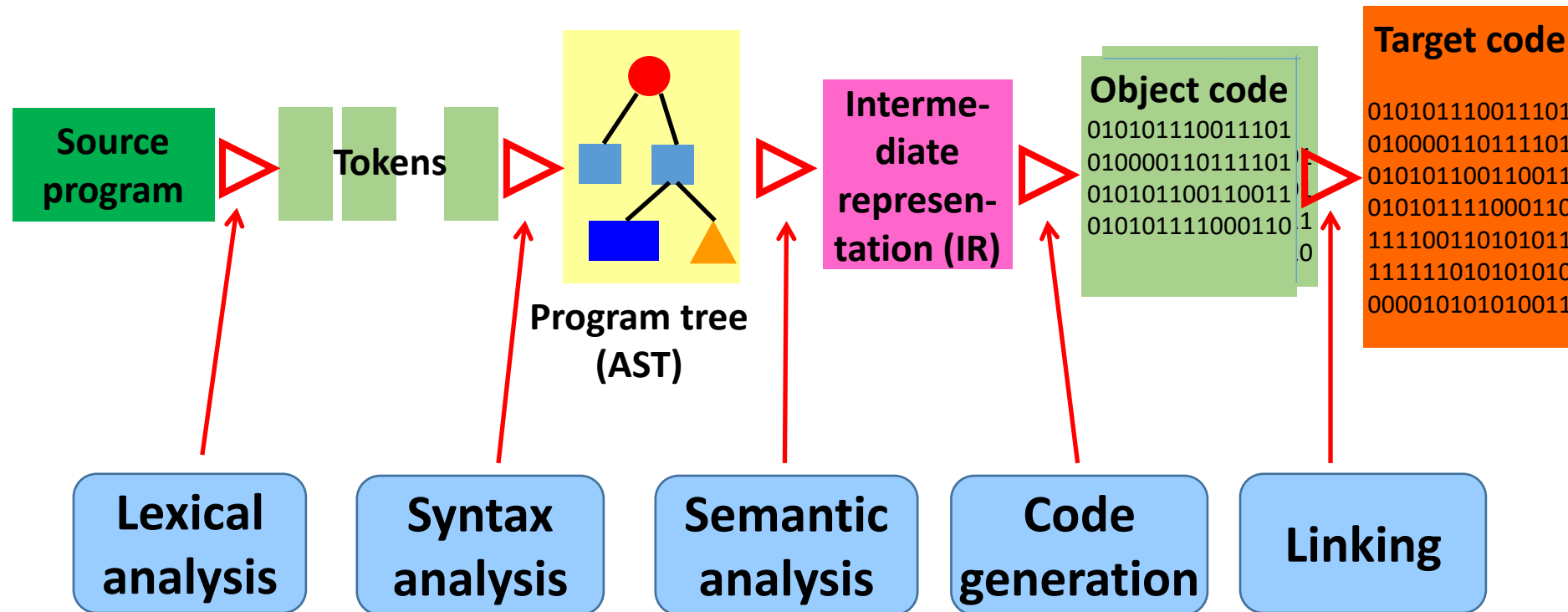
7. Each preprocessing token is converted into a **token**.
8. The resulting tokens are **syntactically and semantically analyzed** and translated as a **translation unit**. Each translated translation unit is examined to produce a list of required instantiations... The definitions of the required templates are located. All the required instantiations are performed to produce **instantiation units**.

Linking

9. All external entity references are **resolved**. Library components are **linked** to satisfy external references to entities not defined in the current translation. All such translator output is collected into a **program image** which contains information needed for execution in its execution environment.

Compilation Data Structures

This is an inverted, **data-centric** view at the compilation process. Compilation is a **sequence of transformations** of a source program.



The Ideal Picture: Modifications (1)

- There may be **several** source units. They can be represented not only as disk files, but may have arbitrary nature (generally, a stream of bytes).
- Lexical analysis may be implemented as a **separate “pass”**, or as a component invoking **“by demand”**, or...
- Lexical analysis can comprise **several “passes”**. Example: C/C++: first preprocessing (macro expansion), and then “true” lexical analysis.
- Lexical & syntax analyses can work either **separately** or **simultaneously** (interacting with each other).
Will be examples for C++ later.

The Ideal Picture: Modifications (2)

- Syntax analysis can be implemented as a single “pass”, or as a **sequence of “passes”** (Java, Scala are examples), and/or can run together with semantic analysis.
- Syntax & semantic analyses can be implemented **as the single stage** (for simple languages like Pascal or Oberon).
- Typically, semantic analysis can include **several “passes”**, or **tree traverses** (see the scheme for C# on the prev. slide); the AST gets modified while each “pass”.

The Ideal Picture: Modifications (3)

- Program tree is build for the complete program or sequentially, for its parts (functions, classes).
- Intermediate representation: either specially designed *compile-time structures*, (e.g. Control Flow Graph - CFG) or *the program tree itself*, or some (other) language.
Examples: lower-level language C--; standard C language as intermediate program representation (see Appendix B)
- Can be several intermediate languages/structures:
see Muchnik book; gcc: GENERIC, GIMPLE, RTL.

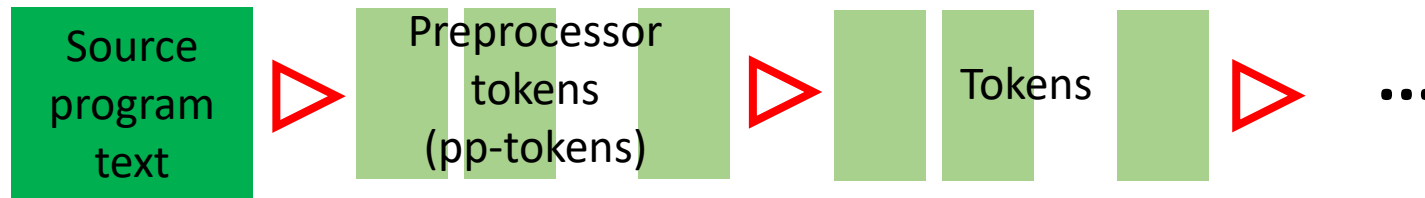
The Ideal Picture: Modifications (4)

- On each compilation phase (from lexical analysis to code generation) external sources can be added to the program: either in the textual form (include files in C/C++), or as precompiled components (libraries).
- Some languages assume/require **linking** stage which is often considered as a standalone phase.
- Code generation phase often (typically) includes some **optimizing sub-phases**.

Compilation Stages:

Lexical Analysis

Components: preprocessor, scanner



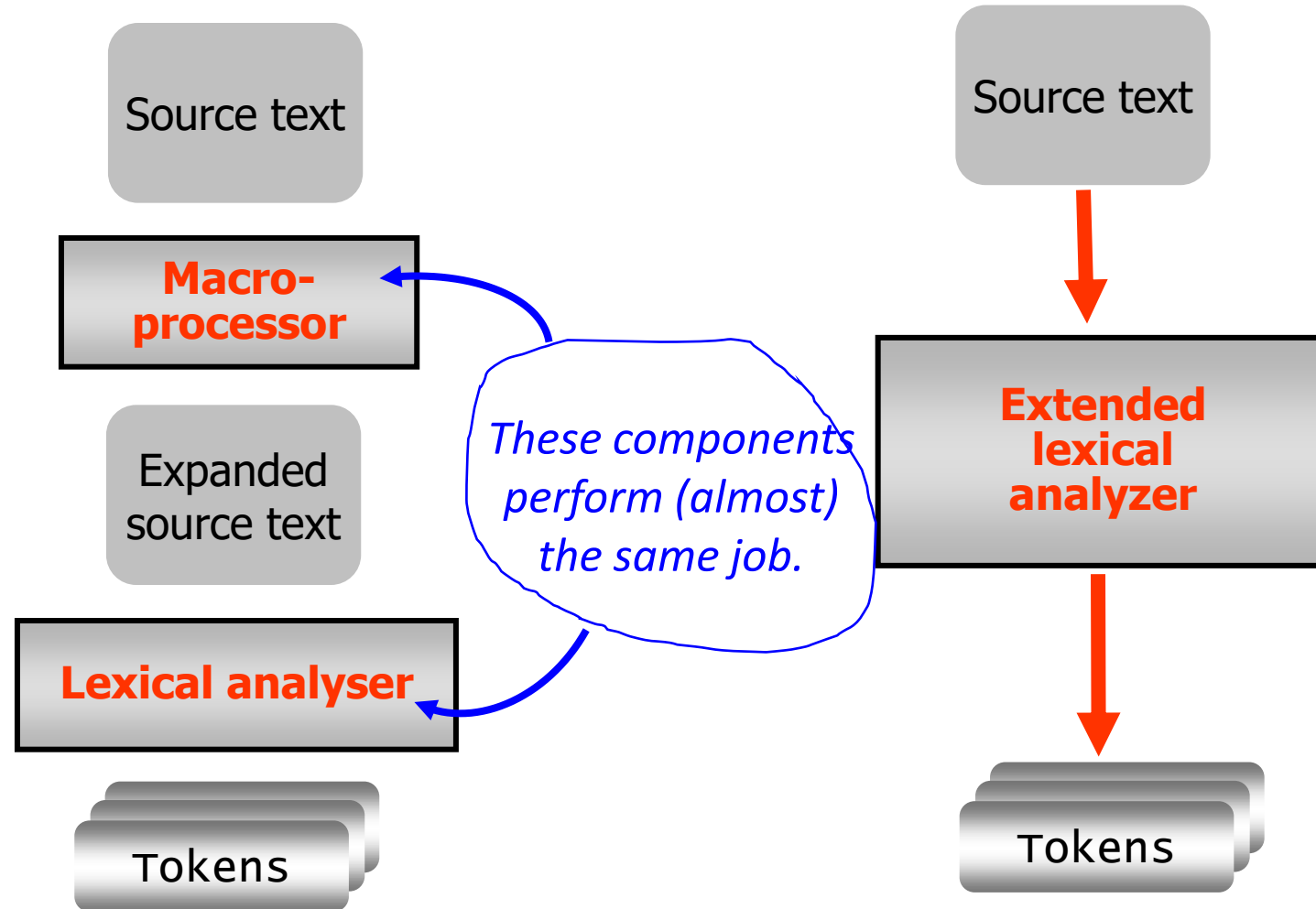
- **Token** is the minimal element of the language alphabet. Examples are operator signs, delimiters, identifiers, keywords, literals.
- **Token representations** in the compiler can be either very simple (e.g., coded by integer values) or be a structure with a set of attributes (see lecture 3 for details).
- How lexical analysis **interacts** with other compiler components: either passing the current token "on demand" or buffering tokens and traversing those buffers (with returns) - see lectures 3-5.

Compilation Stages: Lexical Analysis

Real Example

Implementation alternatives:

- Separately OR together?

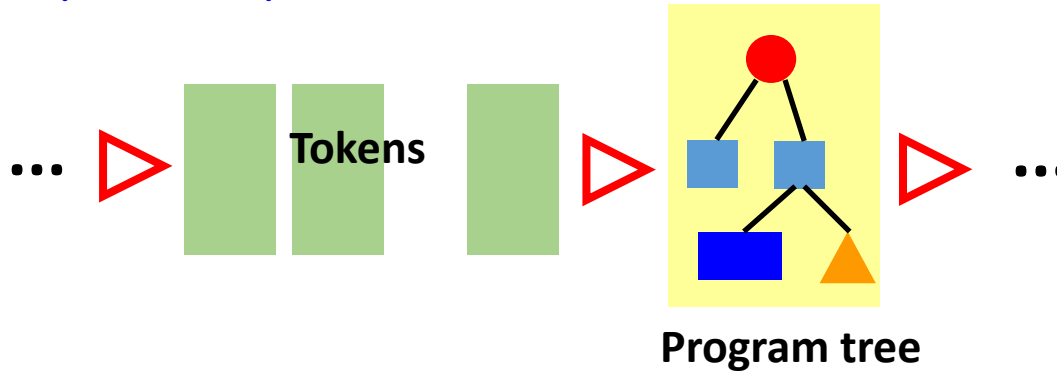


C++ Example

Compilation Stages:

Syntax Analysis

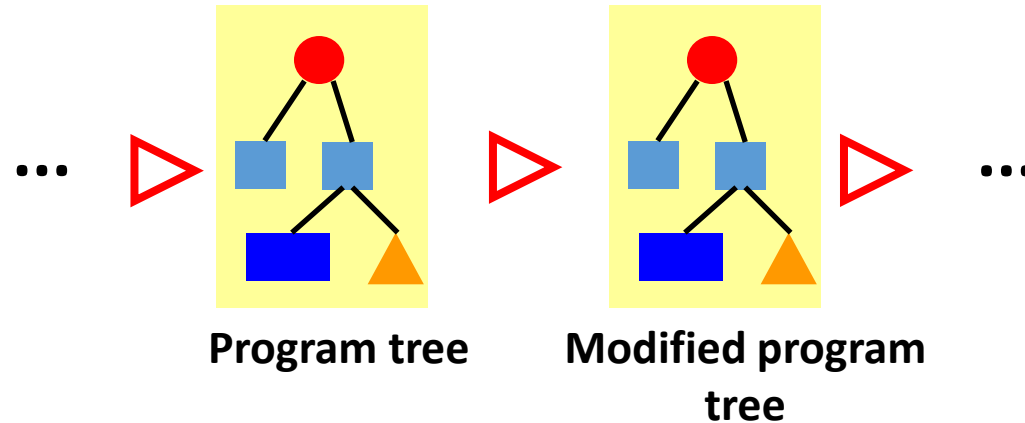
Component: parser



- **Syntax analysis:** Checks the correctness of the syntactical structure of the program in accordance with the source language grammar.
- **The result of the analysis:** An internal **tree-like** representation of the source program where nodes and sub-trees represent structural elements of the source.
- **Parser implementation:** Either hand-written (recursive descent parsing), or automatic parser generation based on a formal language grammar.
- Syntax analysis can be combined with semantic analysis.

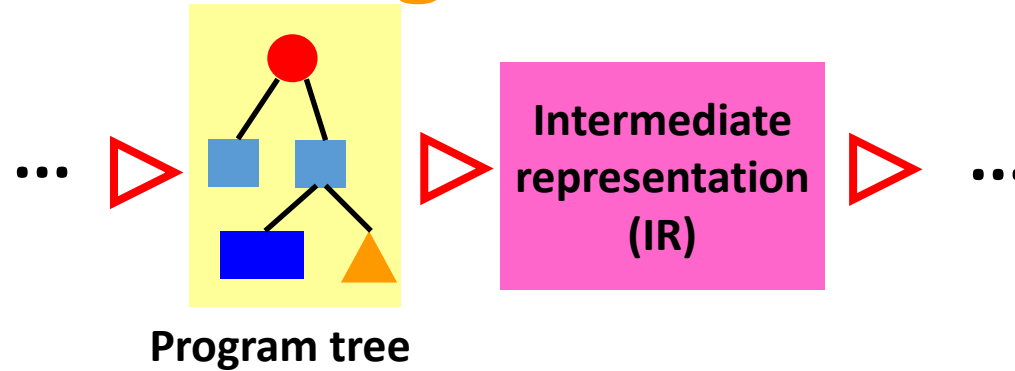
Compilation Stages:

Semantic Analysis



- **Semantic analysis:** Checks semantic correctness of the program against source language definition.
- This is very crucial and the most complicated part of every compiler; the stage typically cannot be automated.
- The result of the analysis: **modified program tree**, perhaps with some extra node attributes («Annotated AST», AAST).
- The semantic analysis is implemented either as a separate component ("analyzer", "validator") or as a "distributed" stage: it can start from the early compilation stages (even from the lexical analysis).

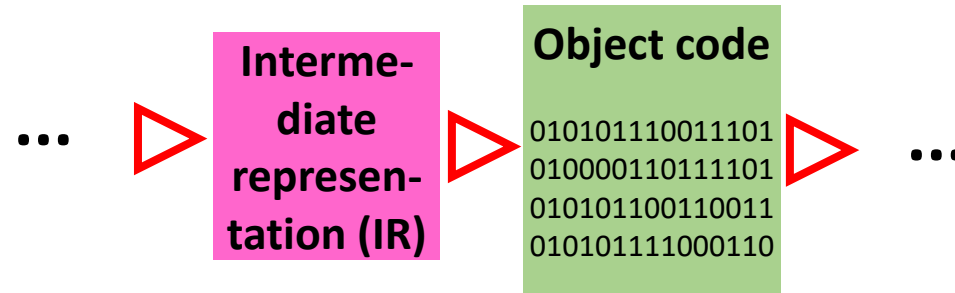
Compilation Stages: IR Generation



- **IR generation** produces a low-level program representation - either for optimization purposes or as a common base for multi-target code generation.
- Often the last tree traversing performs IR generation.
- IR examples:
 - C language ☺; a specially designed low-level language (C--);
 - Assembler language or "generic" assembler language (LLVM bitcode);
 - A proprietary format (e.g., ICode for Scala);
 - Control Flow Graph, CFG.
- Can be dropped for simple cases; (final) code generation can be done directly using AST.

Compilation Stages:

Object Code Generation

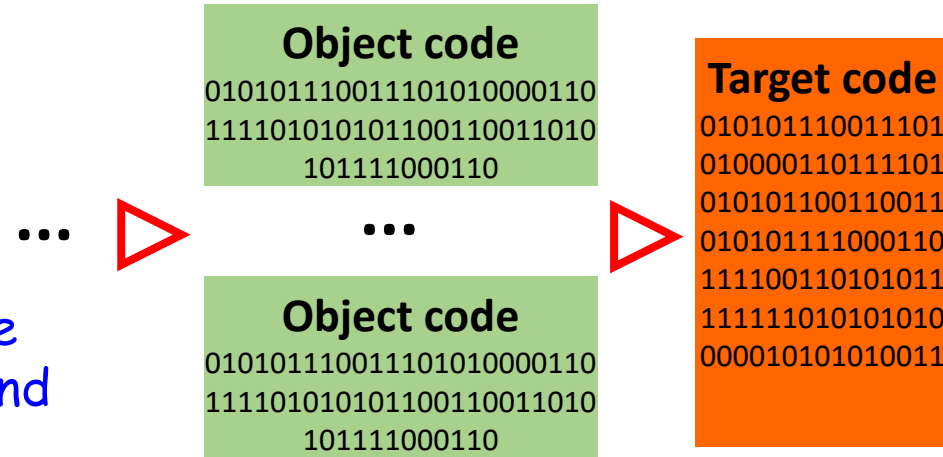


- **Code generation** performs construction of lowest-level machine instructions.
- "Object code" notion is typical to languages like C/C++ where the concept of **separate compilation** is supported. There is no "object code" & linking stage in languages like Java, C#, Oberon.
- Usually, object code is true machine code where addresses are relative (and therefore need to be replaced for real ones), or the code with unresolved external references (which are the subject for resolving).
- Resolving is preformed by **linkage editor** (linker).

Compilation Stages:

Object Code Linking

- Component: linker, linkage editor.
- The linker combines object code parts in the single code image and resolves external references.
- Conceptually this stage is a part of compilation process (see Section 2.2 of the ISO C++ Standard); however, in most cases it is implemented as a standalone component.
- The result of the stage: the sequence of machine instructions with resolved references, ready for execution (or with partially resolved references, ready for further linking).
- For languages that do not support separate compilation, the stage is omitted: «object code» from the previous stage is ready for execution.



Compilation: Two-faced Janus

- Initial stages: language-dependent, and platform-agnostic:

front-end compiler

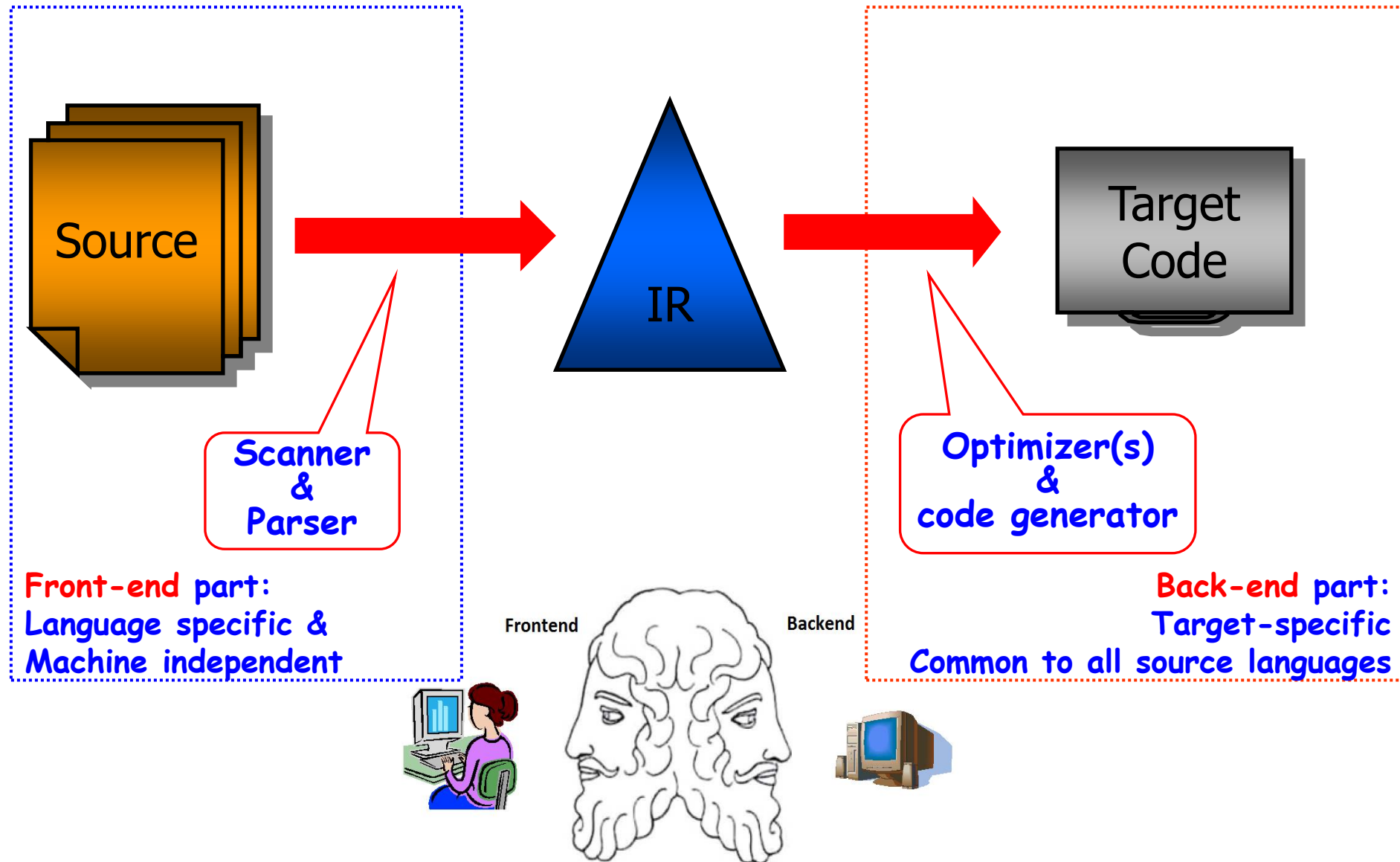
- Final stages: language-independent, and dealing with the specifics of the target machine architecture:

back-end compiler

- Interface ("mediator") between the two compiler components:

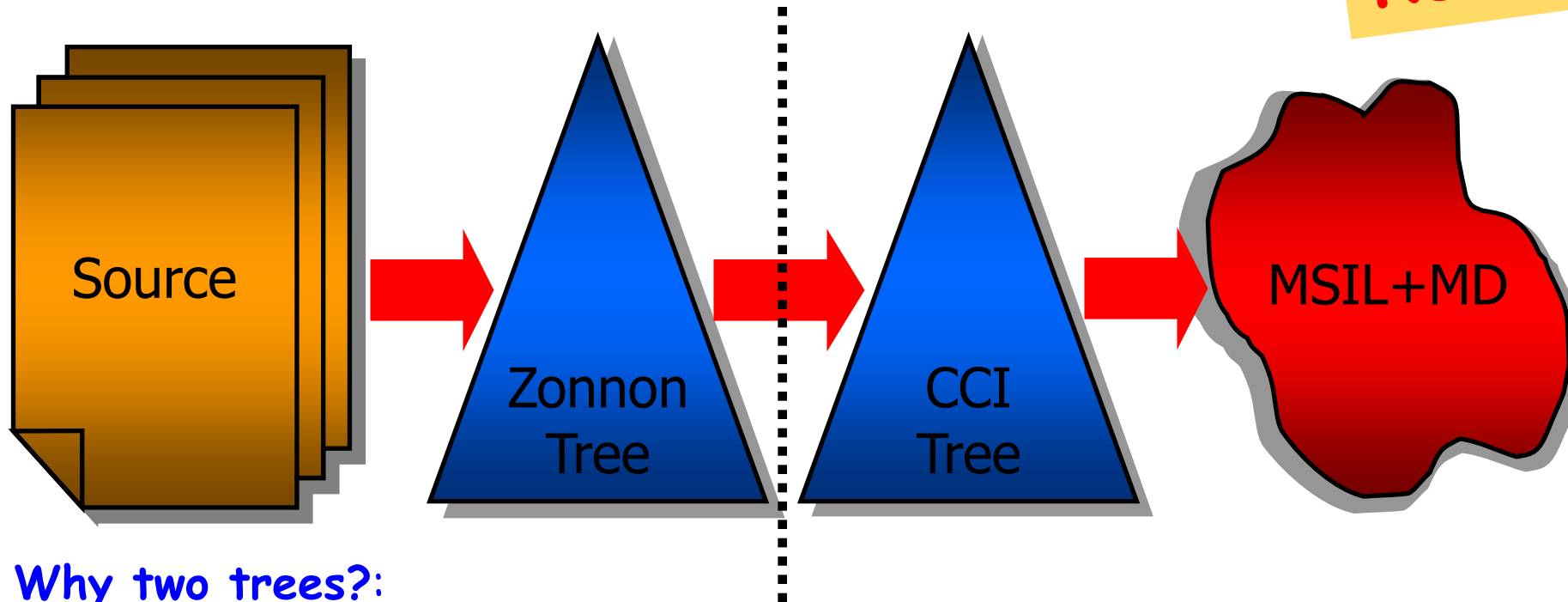
intermediate representation

Compilation: Two-faced Janus



Zonnon Compilation Model

Real Example

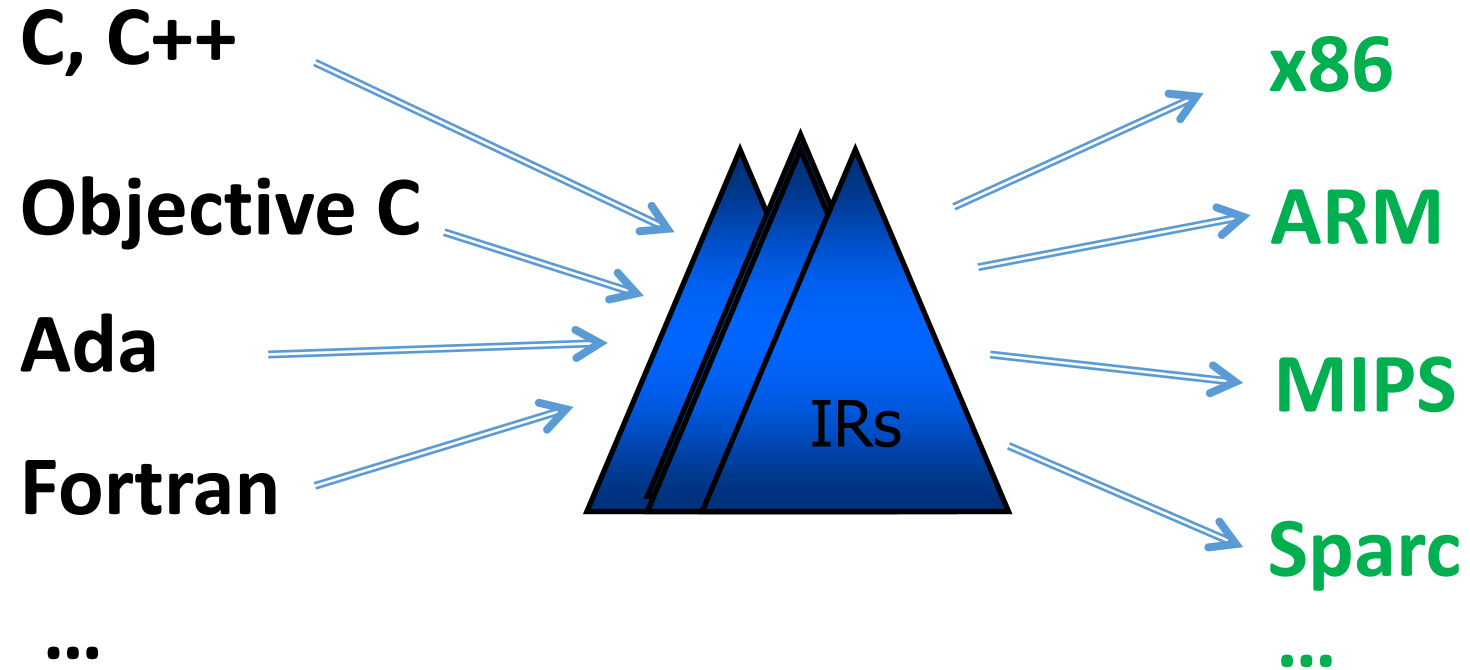


Why two trees?:

- They reflect a **conceptual gap** between Zonnon & CLR
- Zonnon semantic representation **doesn't depend** on CCI & the target platform
- Conversion Zonnon tree -> CCI tree explicitly implements and capsulates **mappings** from Zonnon language notions to the CLR model.

The Advanced Architecture

- Multi-language compilation systems
- Multi-target compilation systems
- Typical example: `gcc` - GNU Compiler Collection



- A newer example: **Clang/LLVM** infrastructure

Program Optimization

- On the stage of lexical and syntax analyses
Big spectrum of optimization techniques.
- On the stage of semantic analysis (while processing AST).
A series of subsequent AST traversing actions. Each traversing can contain some optimizations which are heavily depend on the language semantics.
- On the code generation stage (machine dependent optimizations).
Depend on the target architecture and machine instruction set.
- On the linking stage.
Example: struggling against "code bloat" in C++.

Appendix A

Some historical off-topics

Machine Code & Assembly Language

Off-topic

Mnemonic notation:

ADD 5 7



ADD i j

Binary code of the instruction:

1101010010100111

Hexadecimal code of the instruction:

D4A7



- The ADD instruction denotes the two's complement arithmetic addition. The contents of registers Ri and Rj are arithmetically added, and the result is put into the register Rj.
- The instruction format is as follows:

- Memory state is not considered in the instruction, and the memory state does not change.
- If the addition gives a result which cannot be put into the format specified in the instruction, then **overflow** happens

Assembler code:

.format 32

R5 += R7



Suggested assembly statement for the ADD instruction:

Rj += Ri;

Additional assembly directives specifying the current instruction format:

.format 8; or .format 16; or .format 32;

Compilers: Some History (1)

Off-topic

~~АВТОКОД~~ -> Ассемблер (Язык ассемблера)
~~АВТОКОД 1:1~~ ~~Assembler (Assembly language)~~

~~Программирующая программа~~

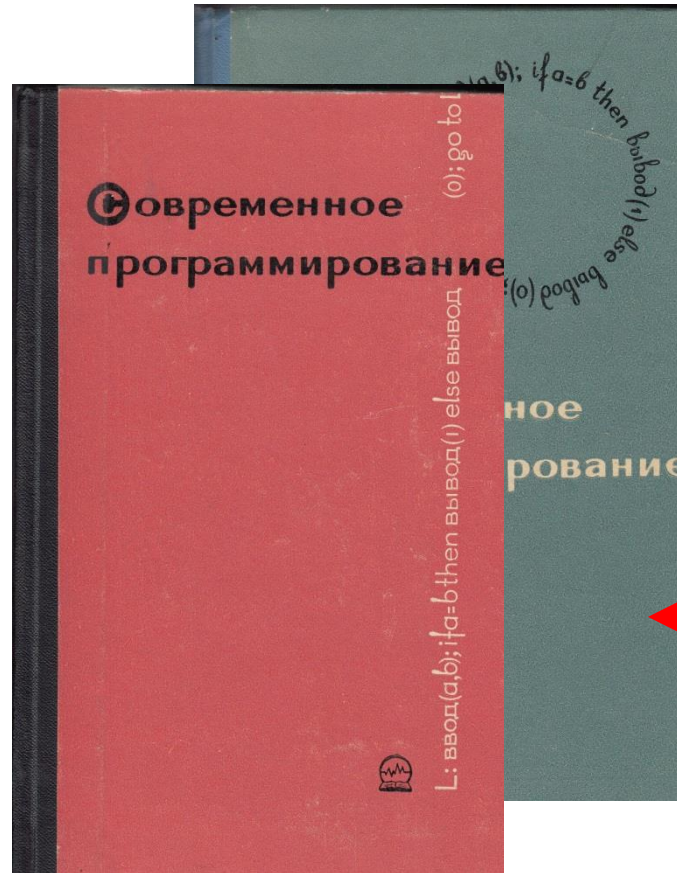
-> Транслятор -> Компилятор
Translator Compiler

Compilers: Some History (2)

Off-topic



1968



1966-1967

Appendix:
More than 300 translators
for Fortran, Cobol, Jovial
etc. for various computers

Compilers: Now and Then

Off-topic

The Past:

- Compiler construction **was** the high level of the programming art, the sign of the top command and/or individual mastery.
- Compilers were among the most complicated software components: one of three cornerstones of computer science (two others are Operating Systems and Databases).

The Present:

- Compiler construction **is** a sort of usual programming job. The average programming qualification became much higher; to make a compiler is under the force of many.
- Software in general becomes much more complicated; for example, implementation of an Internet browser or an XSLT processor is not an easier job than to write a compiler.
- Compiler construction field is now very monopolized. It's hard for a new compiler to compete with existing ones - even if it's better.

Nevertheless! 😊

New languages appear each month!

All this means that if you love compiler construction - you can find a good job 😊

Three main sources of new languages:

- **Big companies** want to have their own development tools: Microsoft, Apple, Google, Facebook, Huawei, Mozilla develop their languages and compilers
- **University professors** want to have their tools to do research and to teach students using more appropriate languages.
- **Individuals** want to become famous and invent their own notation for universal or specific purpose and application domain.
- ...

Go

Rust

Swift

Dart

Pascal

Perl

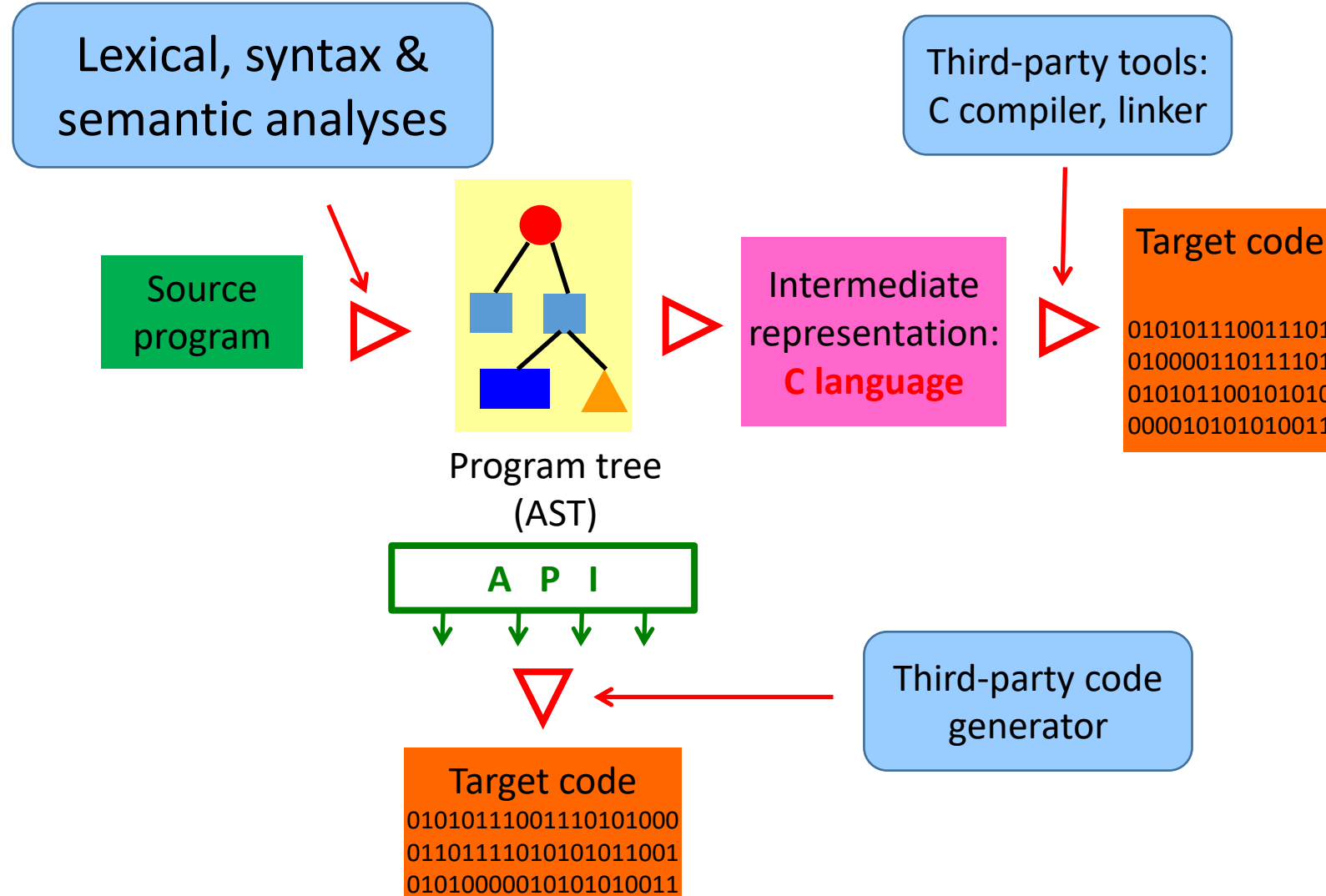
Python

PHP

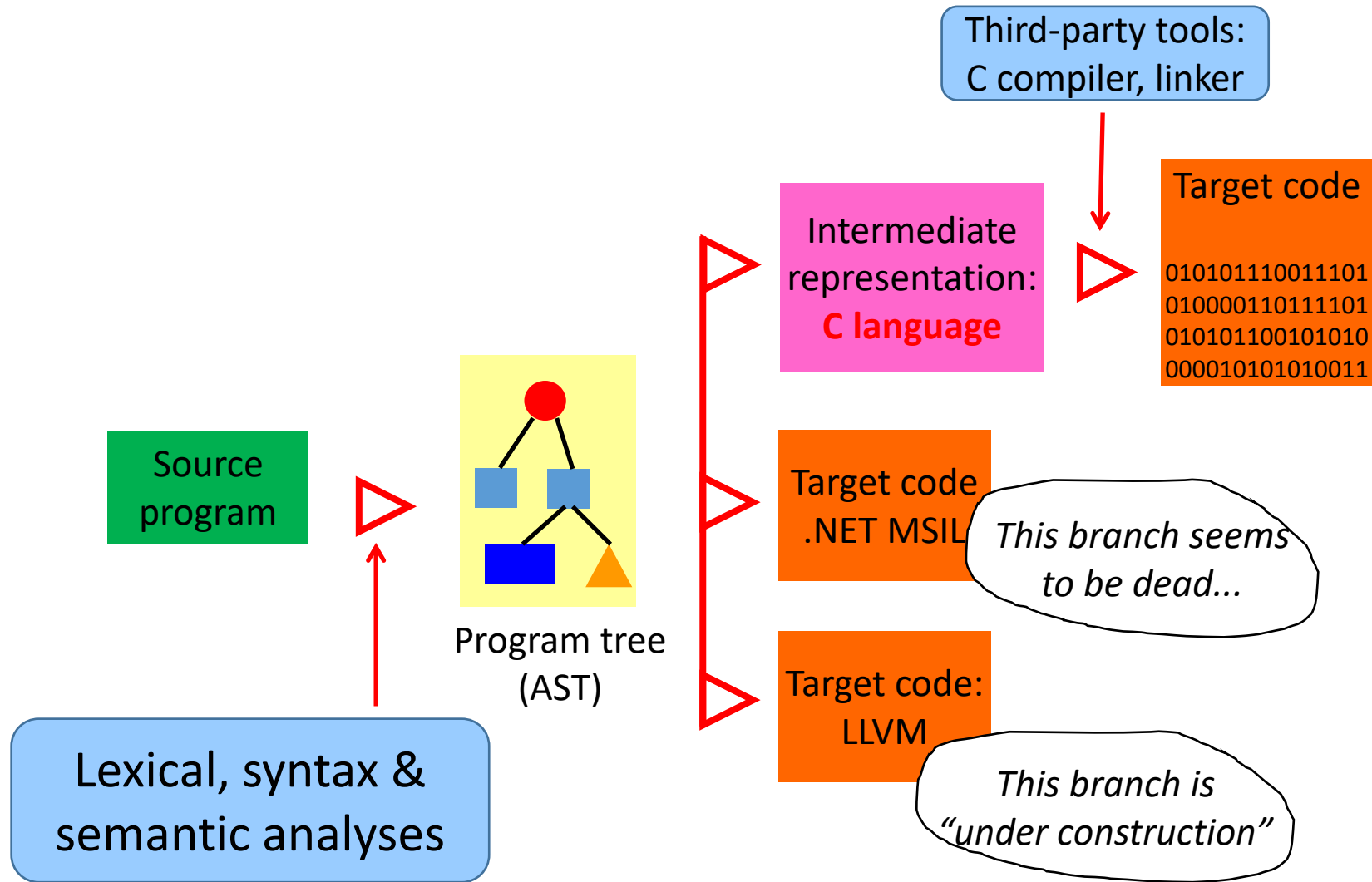
Appendix B

Some real compiler architectures

Compilation Phases & Structures: Edison Design Group C++ Front End



Compilation Phases & Structures: Eiffel



Compilation Phases & Structures: Interstron C++

